PRACTICAL 2

AIM: Implementation of Socket Programming (TCP)

A- A Client Server Based Program using TCP to find if the number entered is prime.

Theory

TCP (Transmission Control Protocol) is a connection-oriented protocol that establishes a connection between a client and a server before exchanging data. In Java, ServerSocket accepts client connections, while Socket enables communication between the client and server. The server checks whether the received number is divisible by any integer from 2 to half of the number and sends the result back to the client.

Working (Stepwise)

1.The server starts and listens on port 8001.

2.The client connects to the server and accepts a number from the user.

3.The client sends the number to the server using DataOutputStream.

4.The server checks whether the number is divisible by any integer in the specified range.

5.The server sends the prime or not-prime result to the client.

6.The client displays the result and closes the connection.

Code 1: tcpServerPrime.java

```
import java.net.*;
import java.io.*;
class tcpServerPrime {
public static void main (String args[]) {
try {
ServerSocket ss = new ServerSocket (8001);
System.out.println(&quot;Pragati&#39;s server has started....&quot;);
Socket s = ss.accept();
DataInputStream in = new DataInputStream(s.getInputStream());
int num = in.readInt();
DataOutputStream out = new DataOutputStream(s.getOutputStream());
boolean flag = false;
for(int i = 2;i&lt;=num/2;++i) {
if(num % i == 0) {
flag = true;
break;
}
}
if(!flag)
out.writeUTF(num + &quot;is a prime number.&quot;);
else
out.writeUTF(num + &quot;is not a prime number.&quot;);
}
catch(Exception e) {
System.out.println(e.toString());
}
}
}
```

Pragati Yadav - CC

Code 2: tcpClientPrime.java

```
import java.net.*;
import java.io.*;
class tcpClientPrime {

MSCIT PART 1_31

public static void main (String args[]) {
try {
Socket cs = new Socket(&amp;quot;LocalHost&amp;quot;,8001);
BufferedReader infu = new BufferedReader(new
InputStreamReader(System.in));
System.out.println(&amp;quot;Enter a Number:&amp;quot;);
int a = Integer.parseInt(infu.readLine());
DataOutputStream out = new DataOutputStream(cs.getOutputStream());
out.writeInt(a);
DataInputStream in = new DataInputStream(cs.getInputStream());
System.out.println(in.readUTF());
cs.close();
}
catch(Exception e) {
System.out.println(e.toString());
}
}
}
```

```
C:\Users\Administrator\Desktop\CC_primeNo>javac tcpServerPrime.java

C:\Users\Administrator\Desktop\CC_primeNo>javac tcpClientPrime.java

C:\Users\Administrator\Desktop\CC_primeNo>java tcpServerPrime
Pragati's server has started....
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator\Desktop\CC_primeNo>java tcpClientPrime
Enter a Number:
7
7is a prime number.

C:\Users\Administrator\Desktop\CC_primeNo>java tcpClientPrime
java.net.ConnectException: Connection refused: connect

C:\Users\Administrator\Desktop\CC_primeNo>java tcpClientPrime
Enter a Number:
12
12is not a prime number.

C:\Users\Administrator\Desktop\CC_primeNo>
```

Pragati Yadav - CC

PRACTICAL 3

AIM: A Client Server Based Program using TCP to find if the number entered is Even or Odd.

THEORY:

TCP is a connection-oriented communication protocol that enables reliable data exchange between a client and a server. In this program, the client sends an integer to the server through a socket connection. The server uses the modulo operator (%) to check divisibility by 2. If the remainder is zero, the number is even; otherwise, it is odd.

Working (Stepwise)

1.The server starts and listens on port 8001.

2.The client connects to the server and accepts an integer from the user.

3.The client sends the integer to the server using DataOutputStream.

4.The server checks whether num % 2 == 0.

5.The server sends the corresponding even or odd message to the client.

6.The client displays the result and closes the connection.

Code 1: tcpserverEven.java

```
import java.net.*;
import java.io.*;
class tcpServerEven {
public static void main (String args[]) {
try {
ServerSocket ss = new ServerSocket (8001);
System.out.println(&quot;Pragati&#39;s server has started....&quot;);
Socket s = ss.accept();
DataInputStream in = new DataInputStream(s.getInputStream());
int num = in.readInt();
DataOutputStream out = new DataOutputStream(s.getOutputStream());
if(num % 2 == 0) {

MSCIT PART 1_31

out.writeUTF(num + &quot;is a Even number.&quot;);
}
else
{
out.writeUTF(num + &quot;is a odd number.&quot;);
}
}
catch(Exception e) {
System.out.println(e.toString());
}
}
}
```

Pragati Yadav - CC

Code 2. TcpCientEven.java

```
import java.net.*;
import java.io.*;
class tcpClientEven {
public static void main (String args[]) {
try {
Socket cs = new Socket(&amp;quot;LocalHost&amp;quot;,8001);
BufferedReader infu = new BufferedReader(new
InputStreamReader(System.in));
System.out.println(&amp;quot;Enter a Number:&amp;quot;);
int a = Integer.parseInt(infu.readLine());
DataOutputStream out = new DataOutputStream(cs.getOutputStream());
out.writeInt(a);

MSCIT PART 1_31

DataInputStream in = new DataInputStream(cs.getInputStream());
System.out.println(in.readUTF());
cs.close();
}
catch(Exception e) {
System.out.println(e.toString());
}
}
}
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator\Desktop\CC_primeNo>java tcpClientEven
Enter a Number:
4
4is a Even number.

C:\Users\Administrator\Desktop\CC_primeNo>
```

```
Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator\Desktop\CC_primeNo>javac tcpServerEven.java

C:\Users\Administrator\Desktop\CC_primeNo>javac tcpClientEven.java

C:\Users\Administrator\Desktop\CC_primeNo>java tcpServerEven
Pragati's server has started....

C:\Users\Administrator\Desktop\CC_primeNo>java tcpServerEven
Pragati's server has started....

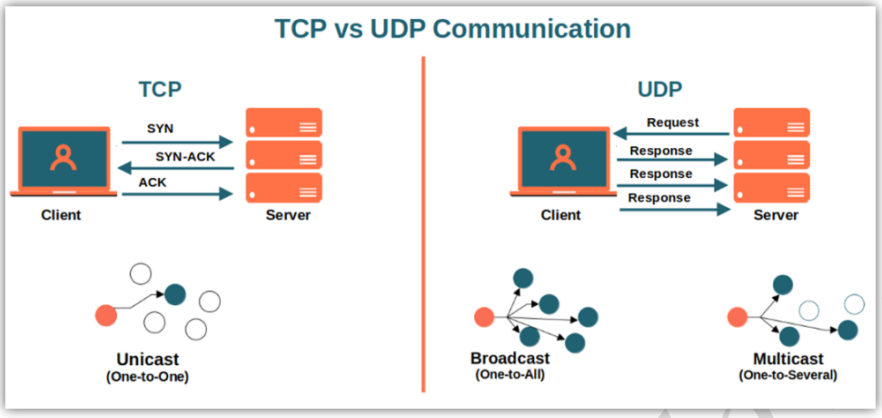
C:\Users\Administrator\Desktop\CC_primeNo>
```

PRACTICAL 4

AIM: A Client Server TCP based Chatting Application

Theory:

Socket programming enables communication between a client and a server over a network. Python's built-in socket module is used to establish communication. The server listens on localhost and port 8000, accepts the client connection, receives messages, and sends replies.



Working (Stepwise).

1.The server creates a socket, binds it to port 8000, and waits for a client connection.
2.The client connects to the server using localhost and port 8000.
3.The client sends messages, which the server receives and displays.
4.The server sends replies to the client.
5.Typing stop terminates the communication.

Code 1: TCPServer

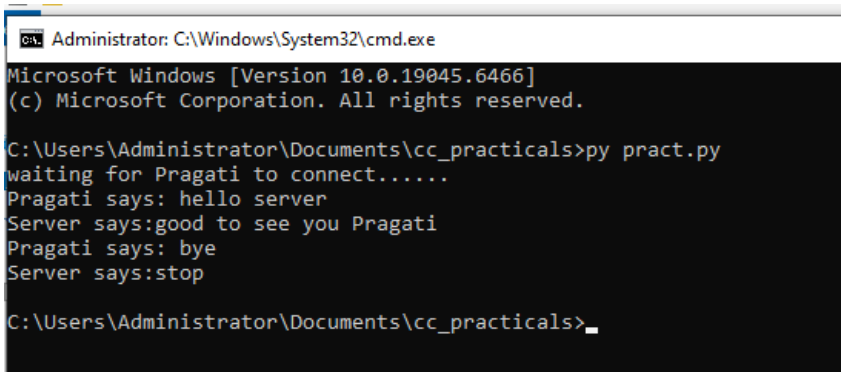
```
import socket
s=socket.socket()
s.bind(('localhost',8000))
s.listen(1)
print("waiting for Pragati to connect......")
c,addr=s.accept()
while True:
    msg=c.recv(1024).decode()
    if msg=="stop":
        break
    print("Pragati says:",msg)
    reply=input("Server says:")
    c.send(reply.encode())
c.close()
s.close()
```

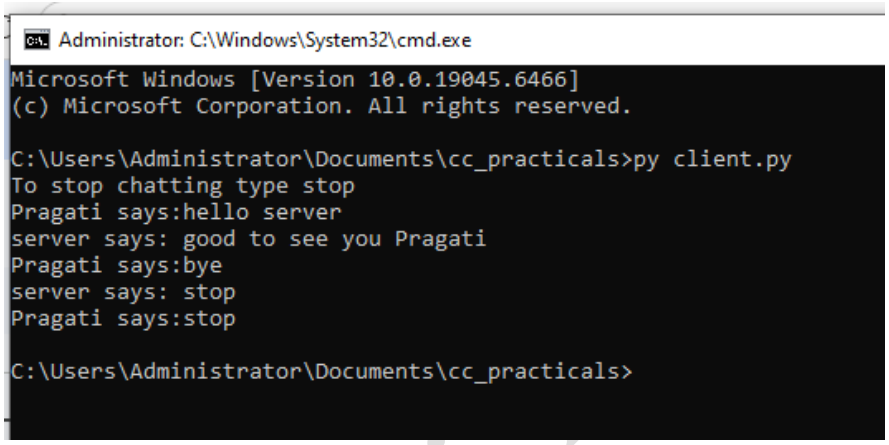
Code 2: TCPClient

```
import socket
s=socket.socket()
s.connect(('localhost',8000))
print("To stop chatting type stop")
while True:
    msg=input("Pragati says:")
```

Pragati Yadav - CC

```
    s.send(msg.encode())
    if msg == "stop":
        break
    print("server says:",s.recv(1024).decode())
s.close()
```





Pragati - CC

PRACTICAL 5

AIM: Implementation of Socket Programming (UDP)

5A. A Client Server based program using UDP to find if the number entered is even or odd.

Theory:

UDP (User Datagram Protocol) is a connectionless communication protocol used to transfer data between a client and server. Python's socket.SOCK_DGRAM is used to create a UDP socket. The server receives a number, checks whether it is even or odd, and sends the result to the client.

Working (Stepwise).

1.The server creates a UDP socket and binds it to localhost and port 2000.

2.The client accepts a number from the user and sends it to the server.

3.The server converts the received number into an integer.

4.The server checks whether the number is divisible by 2 using the modulus operator (%).

5.The server sends the result, even or odd, back to the client, which displays it.

Code 1: UDPServer.py

```
import socket
server_socket = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
server_socket.bind(("localhost",2000))
print("Pragati's server has started....")
data,address = server_socket.recvfrom(1024)
num=data.decode()
print("Received number:",num)


a=int(num)

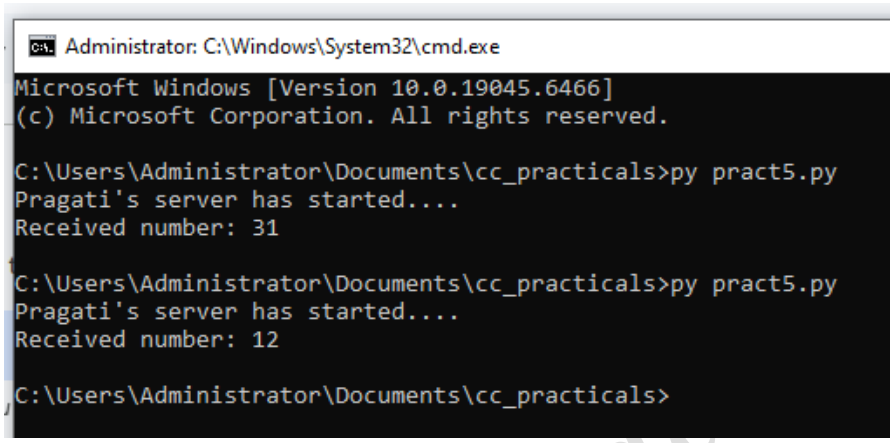

if a%2 == 0:
    result = "Number is even"
else:
    result = "Number is odd"

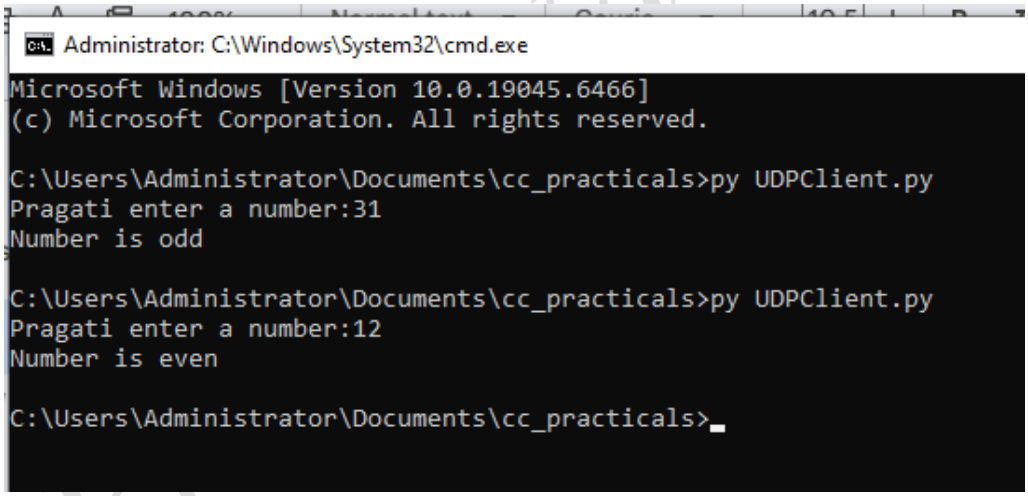

data=result.encode()
server_socket.sendto(data,address)
server_socket.close()
```

Pragati Yadav - CC

Code 2: UDPClient.py

```
import socket
client_socket = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
num = input("Pragati enter a number:")
data = num.encode()
client_socket.sendto(data,("localhost",2000))
data,address = client_socket.recvfrom(1024)
result = data.decode()
print(result)
client_socket.close()
```





5B. To implement a UDP client-server application in Python to calculate the factorial of a given number using server-side processing.

Theory:
UDP enables connectionless communication between a client and server. In this practical, the client sends a number to the server, which calculates its factorial using Python's math.factorial() function and returns the result. Conditional statements and exception handling are used to manage negative numbers and invalid inputs.

Working (Stepwise).
1.The server creates a UDP socket and binds it to localhost and port 2000.
2.The client accepts a number from the user and sends it to the server.
3.The server receives the number and converts it into an integer.
4.The server calculates the factorial using math.factorial() if the number is nonnegative; otherwise, it returns an error message. Invalid inputs are also handled.
5.The server sends the result to the client, which displays it and closes the socket.

Code 1: FactorialServer code:

```
import socket

# Create UDP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Bind to localhost on port 2000
server_socket.bind(("localhost", 2000))

print("Pragati's server has started....")

# Receive data from client
data, address = server_socket.recvfrom(1024)
num = data.decode()
print("Received number:", num)

# Convert to integer
n = int(num)

# Compute factorial
fact = 1
while n > 1:
    fact = fact * n
    n = n - 1

# Convert result to bytes and send back
result = str(fact).encode()
server_socket.sendto(result, address)

server_socket.close()
```

Code2: FactorialClient code:

```
import socket

# Create UDP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Pragati Yadav - CC

```
# Take input from user
num = input("Pragati enter a number: ")

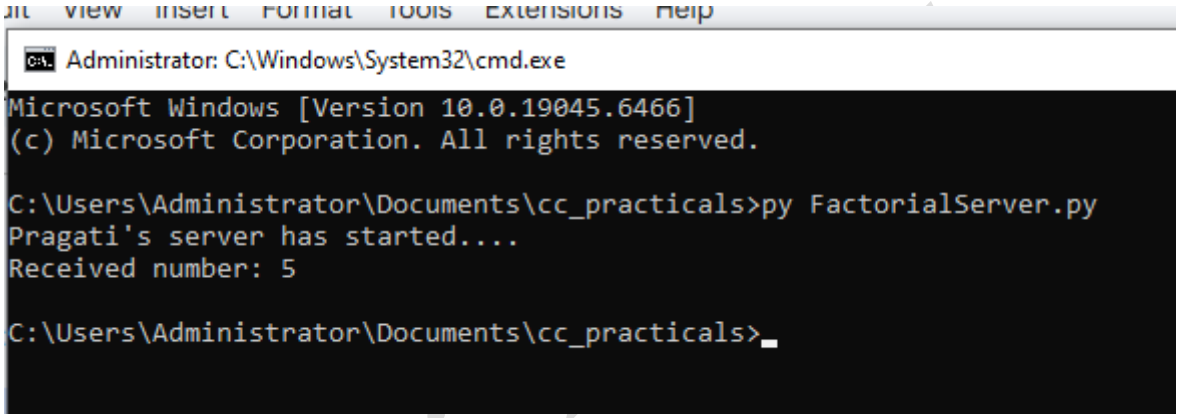
# Convert to bytes
data = num.encode()

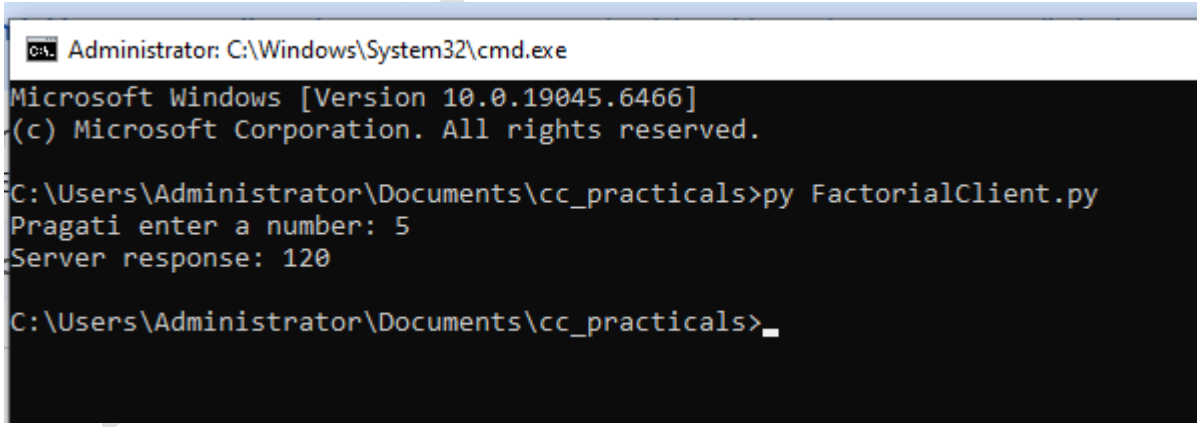
# Send to server (localhost:2000)
client_socket.sendto(data, ("localhost", 2000))

# Receive response from server
data, address = client_socket.recvfrom(1024)
result = data.decode()

# Print result
print("Server response:", result)

# Close socket
client_socket.close()
```





PRACTICAL 6

AIM: Implementation of Socket Programming (RPC)

A. A program to implement simple calculator operations like Addition, Subtraction, Multiplication and Division.

Theory:

Remote Procedure Call (RPC) allows a client to request a function from a server and receive the result. In this practical, UDP sockets are used to send arithmetic operations and numbers to the server, which performs the calculation and returns the result.

Working (Stepwise)

1.The server creates a UDP socket and listens on port 1200.

2.The client sends an operation name and two numbers to the server.

3.The server performs the requested arithmetic operation.

4.The result is sent back to the client on port 1300 and displayed.

RPCServer.py

CODE:

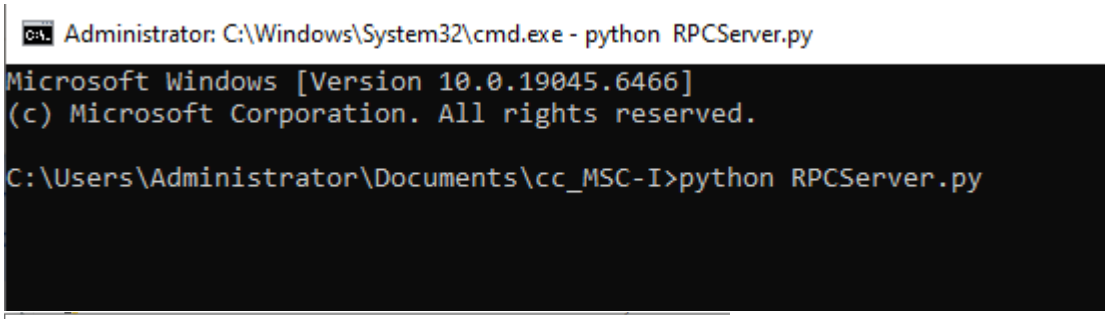
```
import socket
host = socket.gethostbyname(socket.gethostname())
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((host, 1200))
while True:
    data, address = server.recvfrom(1024)
    msg = data.decode().split()
    op, a, b = msg[0], int(msg[1]), int(msg[2])
    if op == "add": res = a + b
    elif op == "sub": res = a - b
    elif op == "div": res = a / b
    elif op == "mul": res = a * b
    server.sendto(str(res).encode(), (host, 1300))
```

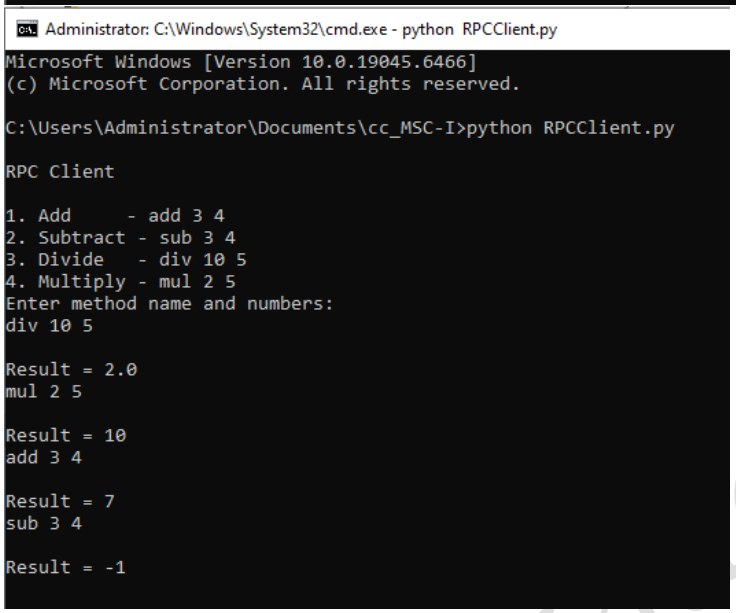
RPCClient.py

CODE:

```
import socket
host = socket.gethostbyname(socket.gethostname())
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.bind((host, 1300))
print("\nRPC Client\n")
print("1. Add     - add 3 4\n2. Subtract - sub 3 4\n3. Divide   - div 10
5\n4. Multiply - mul 2 5")
print("Enter method name and numbers:")
while True:
    data = input()
    client.sendto(data.encode(), (host, 1200))
    result, address = client.recvfrom(1024)
    print("\nResult =", result.decode())
```

Pragati Yadav - CC





6B- To implement an RPC-based program to calculate the square, square root, cube, and cube root of a given number.

Theory:

RPC enables a client to request mathematical operations from a remote server. In this practical, the Python server uses arithmetic operations and the math.sqrt() function to calculate the requested result and return it to the client.

Working (Stepwise)

1.The server creates a UDP socket and waits for client requests on port 1200.

2.The client sends the operation name and a number.

3.The server identifies the operation and calculates the result.

4.The server sends the result to the client, which displays it.

5.Entering q terminates the server.

Pragati Yadav - CC

RPCServer1.py code:

```
import socket
import math


server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("localhost", 1200))


print("RPC Server started....")


while True:
    data, address = server.recvfrom(4096)
    msg = data.decode().split()


    if msg[0].lower() == "q":
        break
    method = msg[0].lower()
    num = int(msg[1])


    if method == "square":
        result=num*num
    elif method == "squareroot":
        result=math.sqrt(num)
    elif method == "cube":
        result=num*num*num
    elif method == "cuberoot":
        result=num**(1/3)


    print("Result:",result)
    server.sendto(str(result).encode(), ("localhost", 1300))
```

RPCClient1.py code:

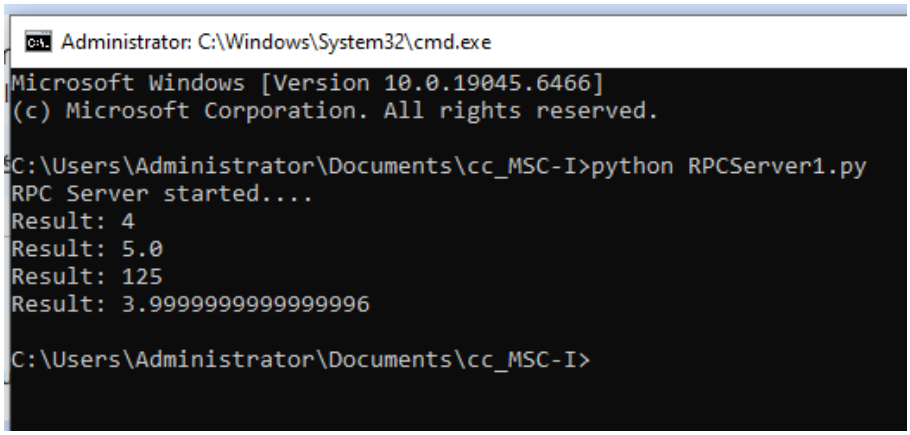
```
import socket

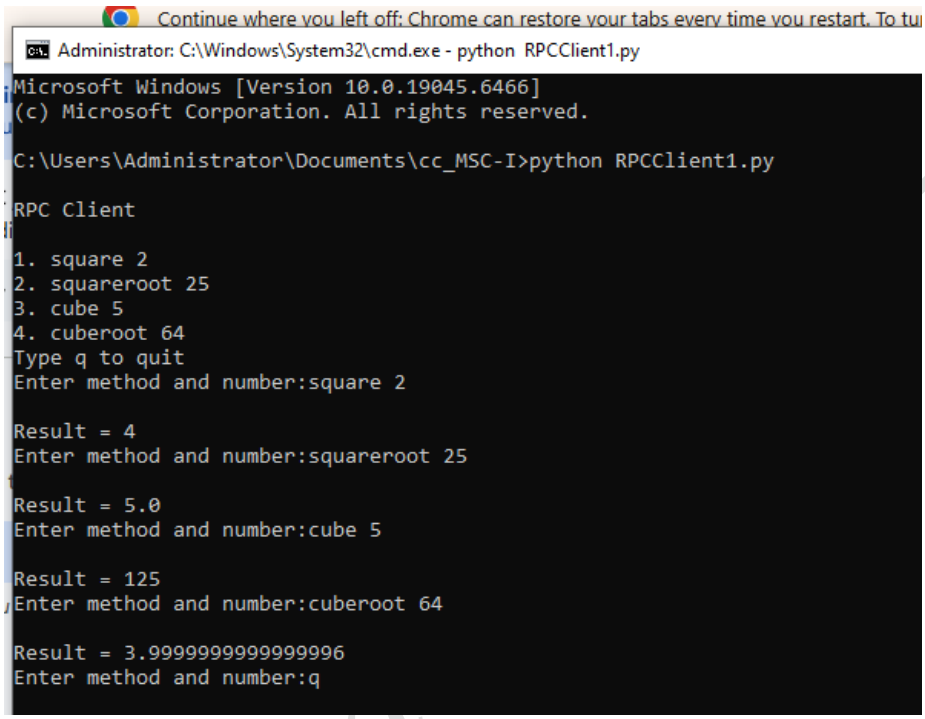
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.bind(("localhost", 1300))

print("\nRPC Client\n")
print("1. square 2")
print("2. squareroot 25")
print("3. cube 5")
print("4. cuberoot 64")
print("Type q to quit")

while True:
    data = input("Enter method and number:")
    client.sendto(data.encode(), ("localhost", 1200))
    result, address = client.recvfrom(4096)
    print("\nResult =", result.decode())
```

Pragati Yadav - CC


Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Administrator\Documents\cc_MSC-I>python RPCServer1.py
RPC Server started....
Result: 4
Result: 5.0
Result: 125
Result: 3.9999999999999996
C:\Users\Administrator\Documents\cc_MSC-I>


Continue where you left off: Chrome can restore your tabs every time you restart. To tu
Administrator: C:\Windows\System32\cmd.exe - python RPCClient1.py
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Administrator\Documents\cc_MSC-I>python RPCClient1.py
RPC Client
1. square 2
2. squareroot 25
3. cube 5
4. cuberoot 64
Type q to quit
Enter method and number:square 2
Result = 4
Enter method and number:squareroot 25
Result = 5.0
Enter method and number:cube 5
Result = 125
Enter method and number:cuberoot 64
Result = 3.9999999999999996
Enter method and number:q

Pragati.cc

PRACTICAL 7

AIM: Implementation of Socket Programming (RMI)

A. An RMI based application program to display current date and time.

Theory:

Java Remote Method Invocation (RMI) allows an object in one Java program to invoke methods of an object running in another program. The server provides a method to return the current date and time, while the client invokes that method remotely.

Working (Stepwise)

1.The remote interface declares the display() method.

2.The server implements the interface and returns the current date and time.

3.The server registers the remote object using Naming.rebind().

4.The client looks up the remote object and invokes display().

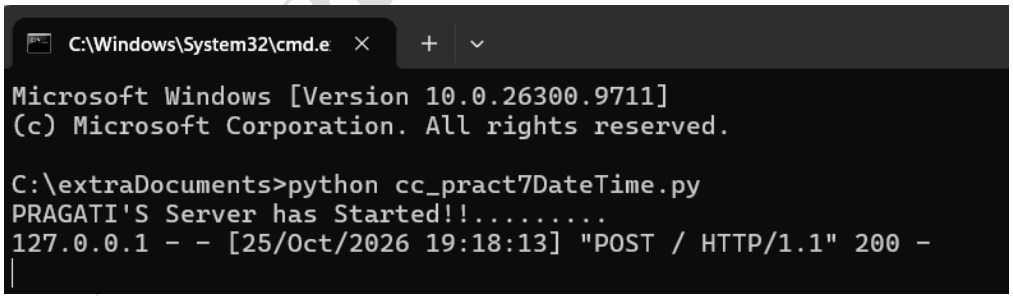
5.The current date and time are displayed on the client.

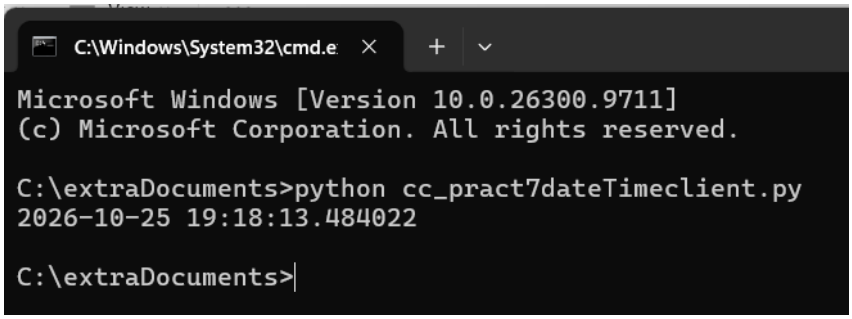
ServerDate.py CODE:

```
from xmlrpc.server import SimpleXMLRPCServer as S
import datetime as d
s = S(("localhost", 8000))
s.register_function(lambda: str(d.datetime.now()), "display")
print("PRAGATI'S Server has Started!!.........")
s.serve_forever()
```

ClientDate.py code:

```
import xmlrpc.client as c
h = c.ServerProxy("http://localhost:8000/")
print(h.display())
```





Pragati Yadav - CC

7B-To implement a client-server application in Python that converts the digits of a given number into their word representations.

Theory:

Socket programming enables communication between a client and a server. In this practical, the server receives a number, converts each digit into its corresponding word, and sends the result back to the client.

Working (Stepwise)

1.The server creates a TCP socket and listens on port 5000.
2.The client accepts a number and sends it to the server.
3.The server converts each digit into corresponding word using a predefined list.
4.The server sends the word representation to the client.
5.The client displays the result.

Code 1: ServerConvert.py

```
import socket
m =
['ZERO','ONE','TWO','THREE','FOUR','FIVE','SIX','SEVEN','EIGHT','NINE']
s = socket.socket()
s.bind(('localhost', 5000))
s.listen(1)
print("Pragati server has started!!......")
while True:
    conn, _ = s.accept()
    data = conn.recv(1024).decode()
    if data:
        ans = " ".join(m[int(c)] for c in data if c.isdigit())
        conn.send(ans.encode())
        conn.close()
```

Code 2: ClientConvert.py

```
import socket
s = socket.socket()
s.connect(('localhost', 5000))
s.send(input("Enter a number: ").encode())
print("Output:", s.recv(1024).decode())
s.close()
```

```
C:\extraDocuments>python cc_pract7DateTime.py
Pragati server has started!!......
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26300.9711]
(c) Microsoft Corporation. All rights reserved.

C:\extraDocuments>python cc_pract7dateTimeclient.py
Enter a number: 31
Output: THREE ONE

C:\extraDocuments>
```

Pragati Yadav - CC

7C- To implement addition of two numbers using XML-RPC in Python.

Theory:

XML-RPC enables remote function calls between a client and server. The server registers an addition function using SimpleXMLRPCServer, and the client calls it using ServerProxy.

Working (Stepwise)

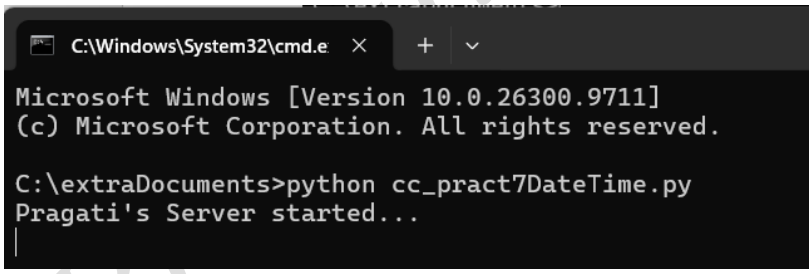
1.The server starts on localhost at port 8000.
2.It registers the add function to return the sum of two numbers.
3.The client accepts two integers.
4.The client calls the remote function and displays the sum.

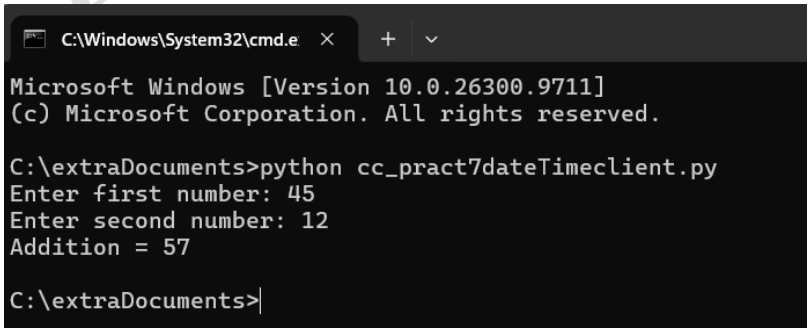
Code 1: serevradd.py

```
from xmlrpc.server import SimpleXMLRPCServer
s = SimpleXMLRPCServer(("localhost", 8000), logRequests=False)
s.register_function(lambda a, b: a + b, "add")
print("Pragati's Server started...")
try: s.serve_forever()
except KeyboardInterrupt: print("\nServer stopping...")
```

Code 2: serevradd.py

```
import xmlrpc.client
s = xmlrpc.client.ServerProxy("http://localhost:8000")
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print(f"Addition = {s.add(a, b)}")
```





Pragati Yadav - CC

PRACTICAL 8

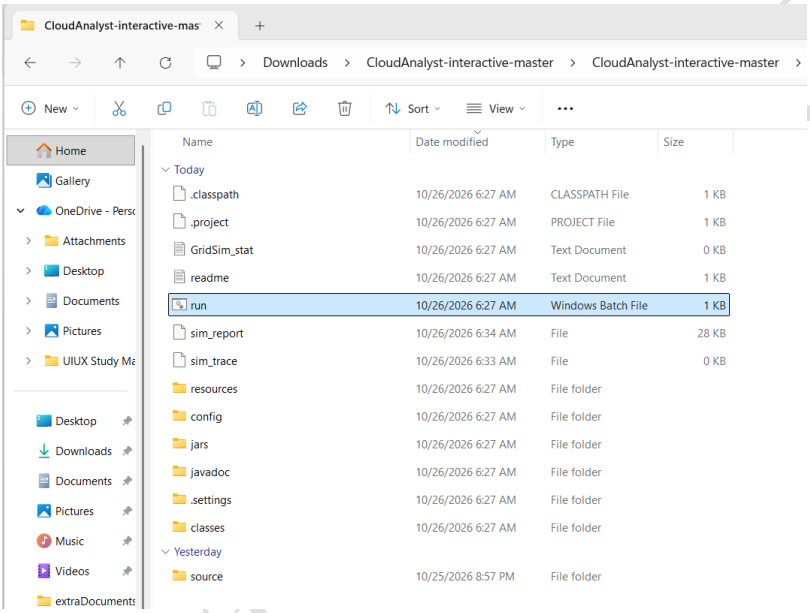
8A Round Robin

Aim: To simulate Round Robin scheduling in CloudSim and analyse virtual machine and cloudlet performance.

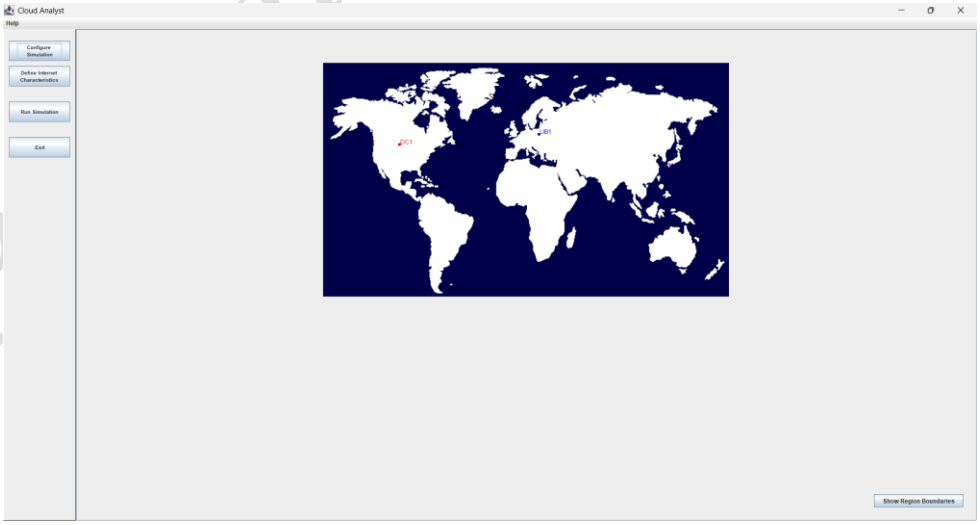
Theory:

Round Robin is a scheduling algorithm that allocates CPU time to processes in cyclic order. CloudSim is used to simulate cloud environments and analyse performance metrics.

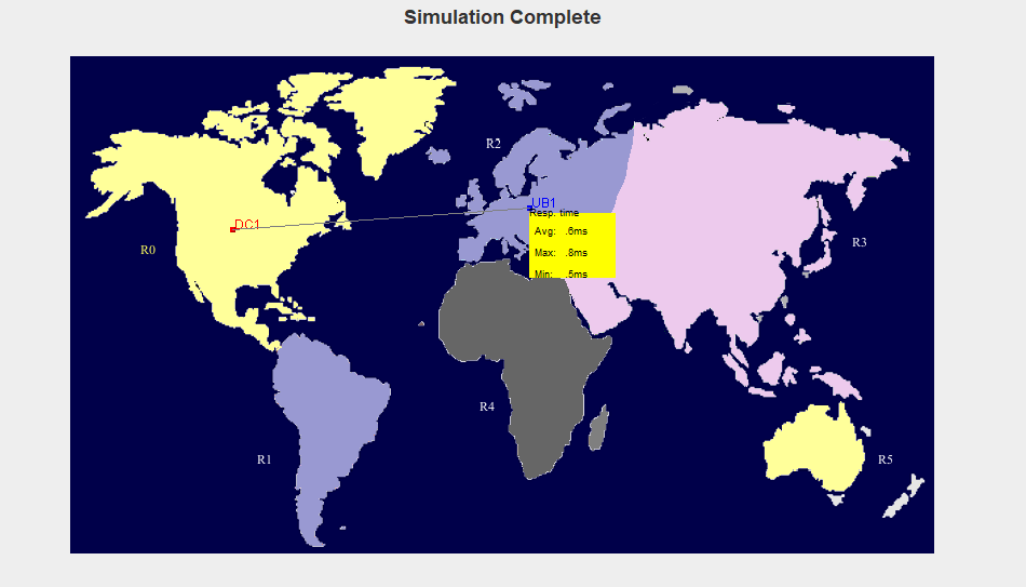
Step 1:Download the GitHub file from:
https://github.com/vondrt4/CloudAnalyst-interactive



Step 2:After that to run it in windows need to double click on run file as selected above.



Step 3: just run simulation and for colors in map there will be one option as show region boundary at the right bottom corner.

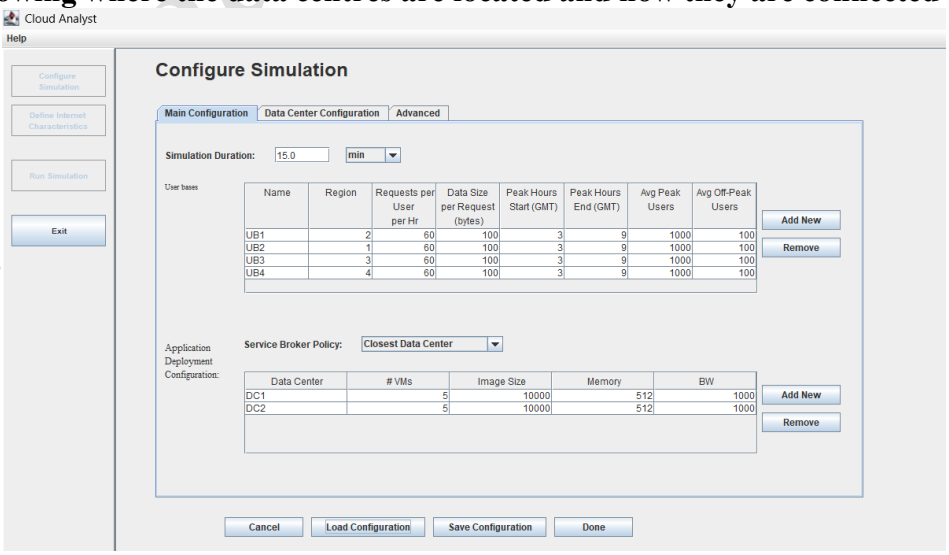


Step 4: Configuration Simulation -> main configuration -> simulation duration (change it to any required time eg: 30/20 min)-> add and remove user basis -> service broker policy (can be changed as per the question) ->

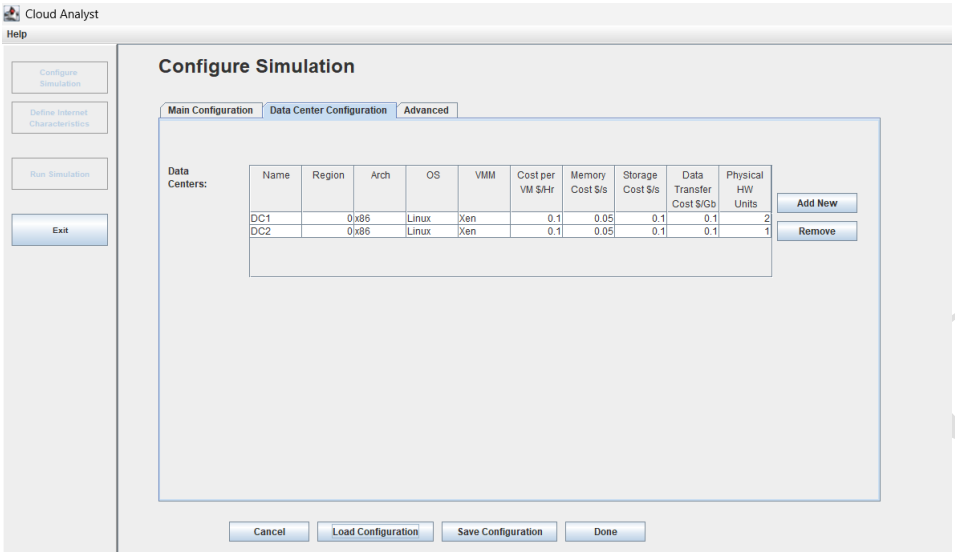
Step 5:Under data centre configuration (add or remove require data centre as for the question can change)

Step 6:Now come under main configuration application deployment configuration we can add the data centre which was previously added under data centre configuration a drop down lease would appear for selecting a data centre

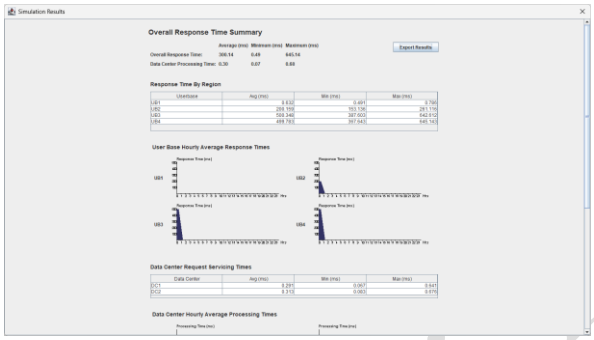
Step 7:Save the configuration by clicking on button save configuration and done
Step 8:Output would appear as in CMD running behind also the map with coloured places showing where the data centres are located and how they are connected



Data center conguration:

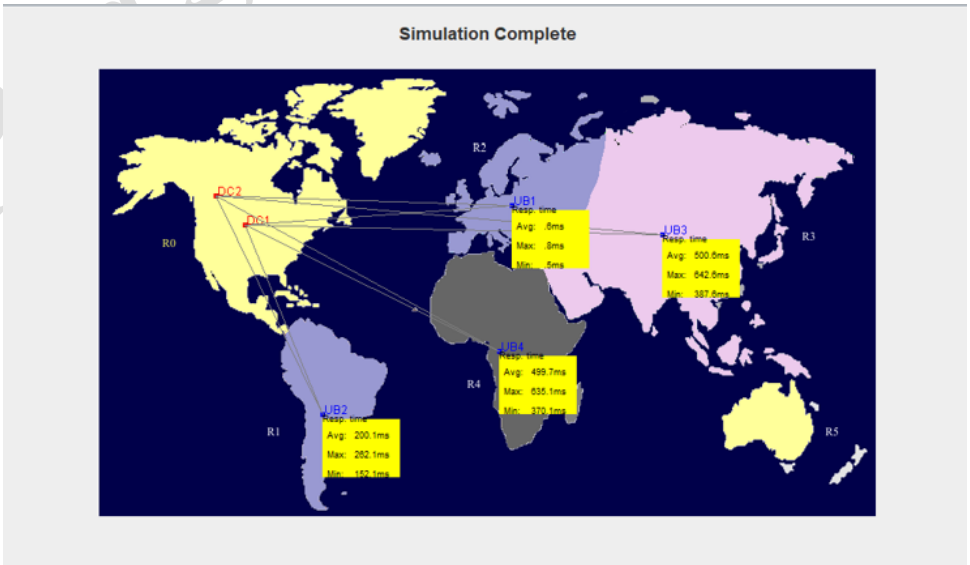


Detailed result:(option will be there at rightbottom corner)

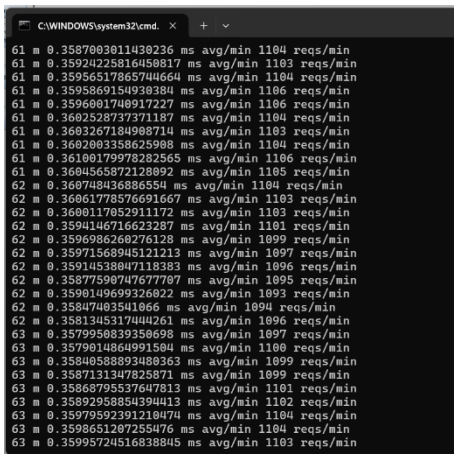


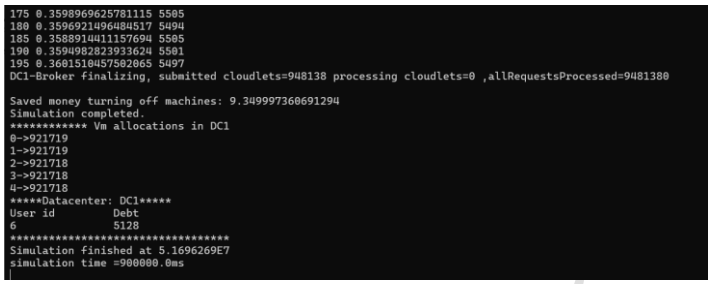


After running the simulation:



CMD output:





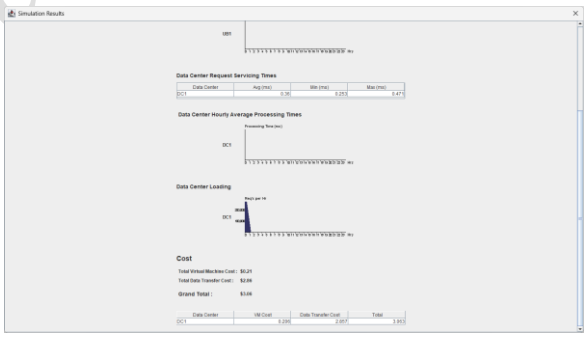
8B Throttle

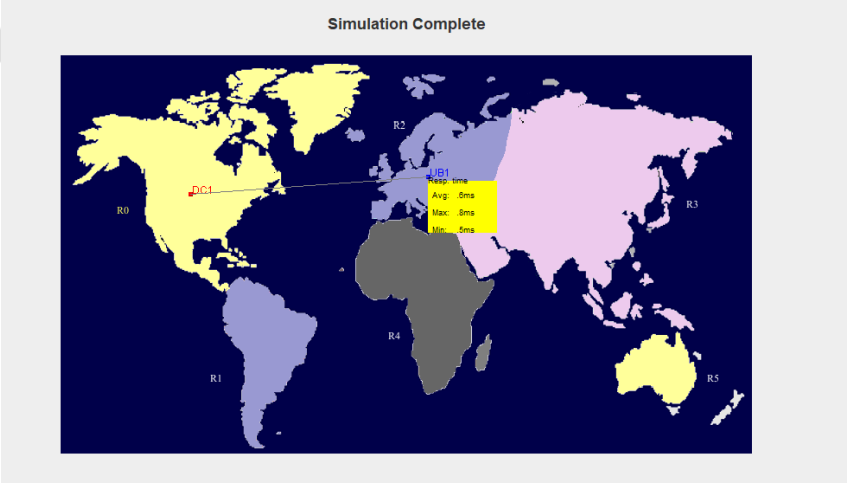
Aim: To simulate Throttled load balancing in CloudSim and analyse task distribution, response time, and cost.

Theory: Throttled load balancing assigns incoming requests to suitable available virtual machines. It uses a virtual machine list to identify an appropriate machine for each request.

Using Round Robin:







```
180 0.3595386719874742 27233
185 0.3595386719874742 27233
190 0.3595386719874742 27233
195 0.3595386719874742 27233
DC1-Broker finalizing, submitted cloudlets=27233 processing cloudlets=0 ,allRequestsProcessed=272330

Saved money turning off machines: 2.3749994099943424
Simulation completed.
********** Vm allocations in DC1
0->11966
1->11965
2->11965
3->11965
4->11965
*****Datacenter: DC1*****
User id         Debt
6               5128
********************************
Simulation finished at 1484029.0
simulation time =8.64E7ms
simulation time =8.64E7ms
```

Using Throttled

